Hosting 5 Websites on One EC2 Server

using Nginx on AWS

A simple step-by-step guide with commands and screenshots

Created by: Mohit Gadilohar

Date: 11 October 2026

Instance: WEB-S-1 | Region: Asia Pacific (Singapore)

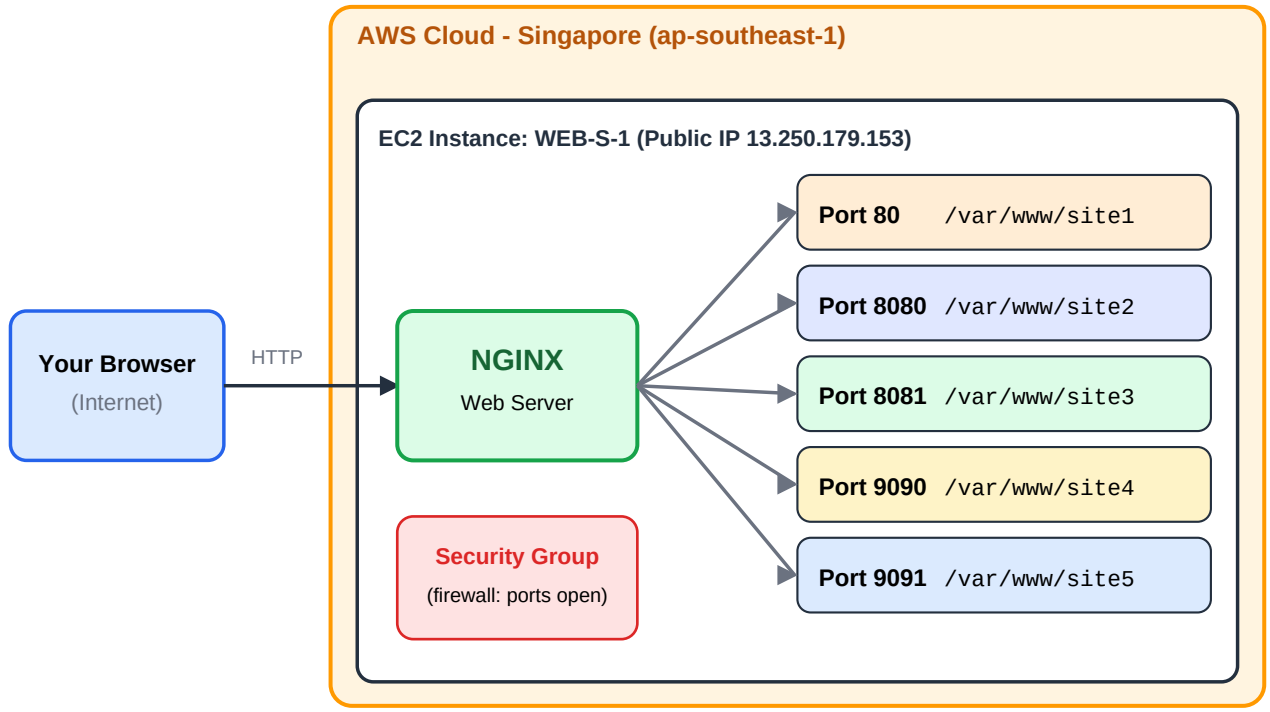
LinkedIn: https://www.linkedin.com/in/mohit-gadilohar-devops

1. What did we build?

We created **one virtual server** on AWS (an **EC2 instance**), installed a web server called **Nginx** on it, and used it to host **five different websites** at the same time. Each website opens on its own **port number**.

Real-life example
Think of the EC2 server as **one big building**. Nginx is the **security guard at the gate**. The five websites are **five shops** inside, each with its own door number (the port). When a visitor says "take me to door 8080", the guard sends them to the right shop.

Architecture diagram



The five websites

Site	Folder on server	Port	Opens at	Page title
Site 1	/var/www/site1	80	http://13.250.179.153	Cloud Engineer
Site 2	/var/www/site2	8080	http://13.250.179.153:8080	DevOps Engineering
Site 3	/var/www/site3	8081	http://13.250.179.153:8081	Linux Server Administration
Site 4	/var/www/site4	9090	http://13.250.179.153:9090	AWS Cloud Services
Site 5	/var/www/site5	9091	http://13.250.179.153:9091	Docker & Containers

About this document
Created by **Mohit Gadilohar**
LinkedIn: https://www.linkedin.com/in/mohit-gadilohar-devops

2. Key words in simple language

Word	Simple meaning
EC2	A virtual computer (server) that you rent from AWS.
Nginx (say "engine-x")	Software that shows your website files to visitors on the internet.
Port	A numbered door on the server. Port 80 is the normal web door; we also use 8080, 8081, 9090 and 9091.
Security Group	The AWS firewall of the server. A port must be allowed here, or nobody can reach it.
Public IP	The internet address of the server: 13.250.179.153. Visitors use this.
Private IP	The address inside AWS network only: 172.31.12.25.
index.html	The main page of a website. Nginx opens it by default.
conf file	A settings file. Ours is /etc/nginx/conf.d/websites.conf.

3. Before you start (checklist)

Item	What we used
AWS account	Logged in to the AWS Console
Region	Asia Pacific (Singapore)
Server (AMI)	Amazon Linux (it uses the yum / dnf package manager)
Instance name	WEB-S-1 (ID: i-01cba865794e6d9df)
Ways to log in	EC2 Instance Connect, CloudShell, or SSH with your key pair

Launching the instance (quick recap)

1. Open **EC2** in the console and click **Launch instance**.
2. Name it **WEB-S-1**, choose **Amazon Linux** and a small free-tier type such as t2.micro or t3.micro.
3. Create or choose a **key pair** (needed for SSH login).
4. In **Network settings**, create a **Security Group** with these **Inbound rules**: SSH (22) and Custom TCP for **80, 8080, 8081, 9090, 9091**.
5. Click **Launch**, wait for *Running*, then connect to the server.

> **Very important**
> If you forget to open the ports in the Security Group, the websites will work inside the server but will **not open in your browser**. Open all five ports (80, 8080, 8081, 9090, 9091) with source 0.0.0.0/0 for this practice setup.

4. Step-by-step process

Every command below was run on the server. In the screenshots the prompt shows `[root@ip-172-31-12-25 ec2-user]#`, which means the commands were run with root (admin) power. If you are the normal **ec2-user**, put `sudo` in front.

1 Update the server

First we update all the installed software, so the server is fresh and safe.

```
yum update -y
```

Why do we use this?
yum is the package manager of Amazon Linux (like the Play Store for servers). **update** brings every installed package to the latest version with security fixes. **-y** answers "yes" automatically, so the command does not stop to ask. We do this first because old packages can cause errors and are a security risk.

Expected result: A list of packages is updated and then you see *Complete!*

2 Install Nginx

This downloads and installs the Nginx web server. The `-y` option means "say yes to every question".

```
yum install nginx -y
```

Why do we use this?
A new EC2 server does not come with a web server. **install nginx** downloads Nginx from the AWS repository and sets it up, so the server can show websites. Without this step, there is nothing to answer a browser's request.

Expected result: Nginx is installed and the screen ends with *Complete!*

Tip
In the history, line 2 shows a typo (`nginx-y` without a space). It failed, so the command was typed again correctly in line 3. Typos are normal. Just run the command again.

3 Create 5 folders (one for each website)

Each website gets its own folder inside /var/www. The `-p` option creates the folder even if the parent folder is missing.

```
mkdir -p /var/www/site1
mkdir -p /var/www/site2
mkdir -p /var/www/site3
mkdir -p /var/www/site4
mkdir -p /var/www/site5
```

Why do we use this?
mkdir means "make directory" (folder). **-p** creates missing parent folders and does not give an error if the folder already exists. **/var/www** is the standard place for website files on Linux. One separate folder per site keeps the content apart, so Nginx can connect each port to its own folder.

Expected result: No message is shown. This means success. You can check with `ls /var/www`.

4 Create an empty index.html in every folder

The `touch` command creates an empty file. This file will become the home page of each website.

```
touch /var/www/site1/index.html
touch /var/www/site2/index.html
touch /var/www/site3/index.html
touch /var/www/site4/index.html
touch /var/www/site5/index.html
```

Why do we use this?
touch creates a new empty file quickly. We create the file first so the name and location are correct. The name **index.html** matters: Nginx opens a file with this name by default when a visitor arrives.

Expected result: No message is shown. The 5 empty files are now created.

5 Write the website content with the vi editor

Open each file in the **vi** editor and paste your HTML code. Repeat for site1 to site5 with different content.

```
vi /var/www/site1/index.html
vi /var/www/site2/index.html
vi /var/www/site3/index.html
vi /var/www/site4/index.html
vi /var/www/site5/index.html
```

Why do we use this?
vi is a text editor that runs inside the terminal. A server has no mouse or desktop, so we edit files with a terminal editor. vi is installed on almost every Linux server, so it is a skill you will use again and again.

Expected result: Each file opens in the editor.

Tip
vi basics: press **i** to start typing, paste your code, press **Esc**, type **:wq** and press **Enter** to save and exit.

Example HTML (Site 1)

```
<!DOCTYPE html>
<html>
<head>
  <title>Cloud Engineer</title>
</head>
<body style="background:#e8f3ff; font-family:Arial; text-align:center;">
  <h1 style="color:#0b5f8a;">Cloud Engineer</h1>
  <h2>Welcome to Cloud Computing</h2>
  <p>Learn AWS, EC2, S3, VPC and IAM.</p>
  <p>Build scalable and reliable cloud infrastructure.</p>
</body>
</html>
```

Use a different title, heading, text and colour in every site, so you can easily see which website is opening.

6 Create the Nginx configuration file

This is the most important step. Here we tell Nginx: *"listen on this port and show files from this folder"*. We keep all five websites in one file inside /etc/nginx/conf.d/.

```
sudo vi /etc/nginx/conf.d/websites.conf
```

Why do we use this?
sudo means "run this as administrator". The folder **/etc/nginx/conf.d/** is protected, so a normal user cannot save files there. Nginx automatically reads every .conf file in this folder, so we do not need to touch the main nginx.conf file. **vi** is used here to write the settings.

Typical content of websites.conf

```
server {
    listen 80;
    server_name _;
    root /var/www/site1;
    index index.html;
}
server {
    listen 8080;
    server_name _;
    root /var/www/site2;
    index index.html;
}
server {
    listen 8081;
    server_name _;
    root /var/www/site3;
    index index.html;
}
server {
    listen 9090;
    server_name _;
    root /var/www/site4;
    index index.html;
}
server {
    listen 9091;
    server_name _;
    root /var/www/site5;
    index index.html;
}
```

Line	Meaning
`server { ... }`	One block = one website.
`listen 8080;`	The port (door number) of this website.
`server_name _;`	Accept any name or IP address.
`root /var/www/site2;`	The folder that holds the website files.
`index index.html;`	The page to open first.

Note
The screenshots do not show the inside of websites.conf, so the code above is the standard pattern that matches the five folders and ports used here. Your own file may look slightly different, which is fine as long as each port points to the correct folder.

If port 80 still shows the Nginx test page
The default Nginx settings file (/etc/nginx/nginx.conf) may also have a server block on port 80. Comment that block out (put # in front of each line) or add `default_server` after `listen 80` in your block, then test and restart again.

7 Test the configuration

Before restarting, always check that the settings file has no mistakes.

```
sudo nginx -t
```

Why do we use this?
nginx -t (t = test) checks the settings file for mistakes **without** changing the running server. It is a safety check: if there is an error, your existing websites do not go down.

Expected result: *syntax is ok* and *test is successful*. If you see an error, fix the line it mentions in websites.conf.

Tip
Never restart Nginx without testing. A single missing semicolon (;) can stop all your websites.

8 Restart Nginx

Restarting makes Nginx read the new settings file.

```
sudo systemctl restart nginx
```

Why do we use this?
systemctl is the tool that starts, stops and restarts services on Linux. **restart** stops Nginx and starts it again, so it reads the new settings and opens the new ports. **sudo** is needed because managing services is an admin job.

Expected result: No message means success.

Tip
To make Nginx start automatically every time the server reboots, also run: `sudo systemctl enable nginx`

9 Check that Nginx is listening on all 5 ports

The `ss` command shows network connections. We ran it before and after the restart to see the difference (history lines 21 and 23).

```
sudo ss -intp
```

Why do we use this?
ss (socket statistics) shows which programs are using which network ports. It is the modern replacement of the older **netstat** command. Flags: **-i** = extra connection details, **-n** = show numbers (not names), **-t** = TCP only, **-p** = show the program name (needs sudo). We run it before and after the restart to see that Nginx really opened the new ports.

Expected result: You can see nginx working on ports 80, 8080, 8081, 9090 and 9091. In our screenshot, a line like `SYN-RECV ... 172.31.12.25:80` is a visitor's browser just connecting to port 80.

Tip
For a cleaner view of only the listening ports, use: `sudo ss -ltnp | grep nginx` (l = listening, t = TCP, n = numbers, p = program).

10 Open every website in the browser

On your laptop, open a browser and type the server's **public IP** followed by the port. Use Incognito mode so the browser does not show an old saved page. A *Not secure* label is normal, because we use http without an SSL certificate.

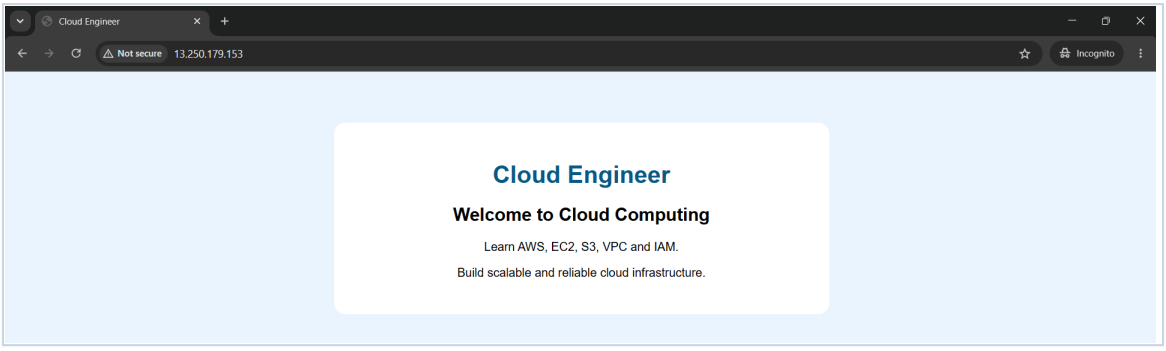
```
http://13.250.179.153
http://13.250.179.153:8080
http://13.250.179.153:8081
http://13.250.179.153:9090
http://13.250.179.153:9091
```

Why do we use this?
No Linux command is needed here. The browser is the final test: if the page opens, then the server, Nginx, the Security Group and the website files are all working together.

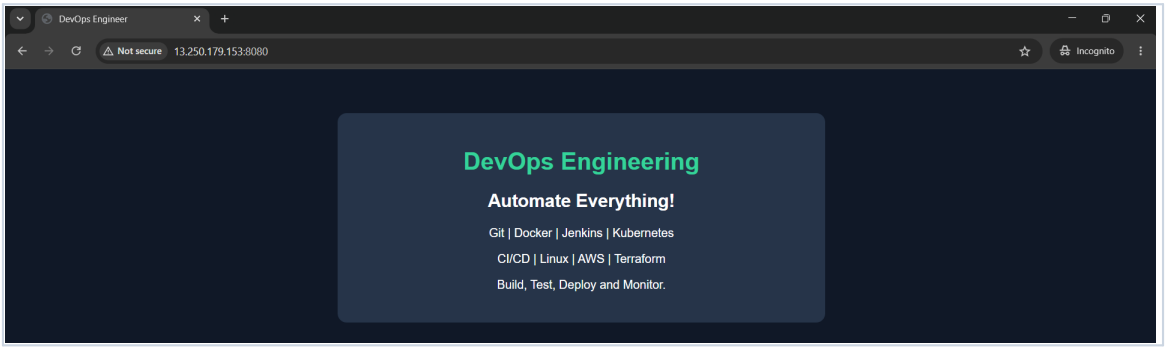
Expected result: Five different websites open, as shown in the next section.

5. Output: our 5 websites are live!

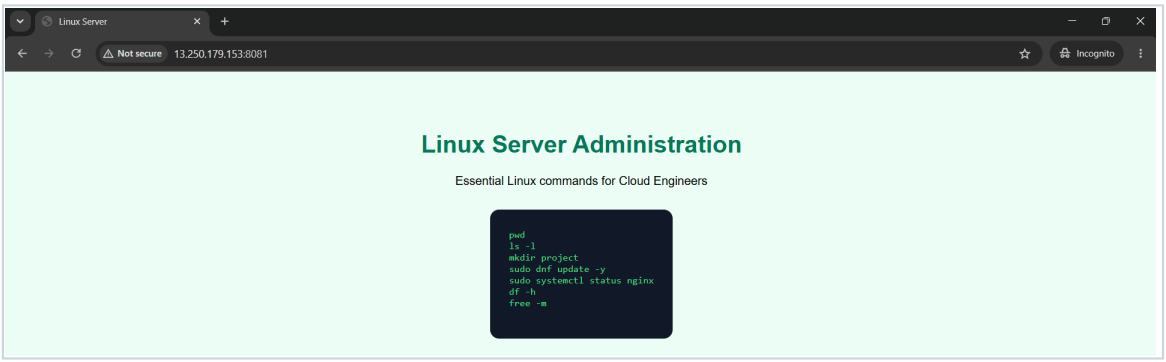
All five websites are opened using the same public IP **13.250.179.153**, only the port changes.



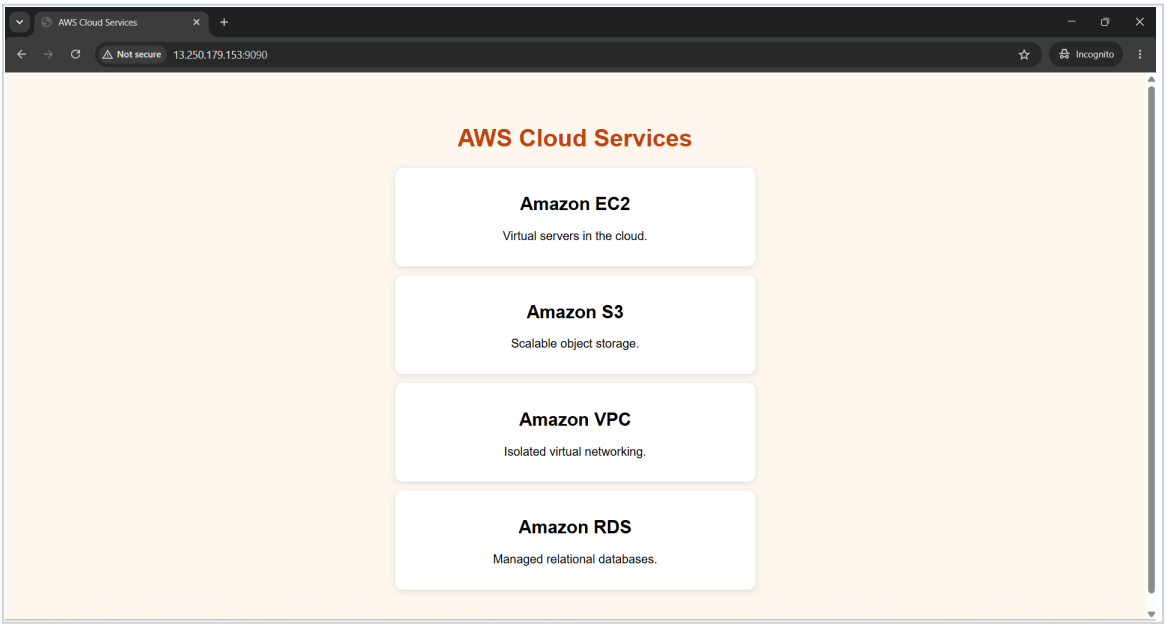
Website 1 - Port 80 - http://13.250.179.153 - "Cloud Engineer"



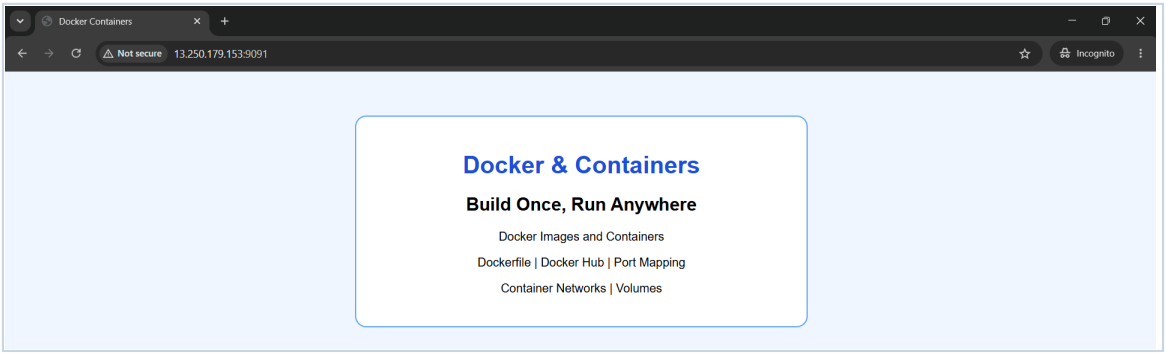
Website 2 - Port 8080 - http://13.250.179.153:8080 - "DevOps Engineering"



Website 3 - Port 8081 - http://13.250.179.153:8081 - "Linux Server Administration"



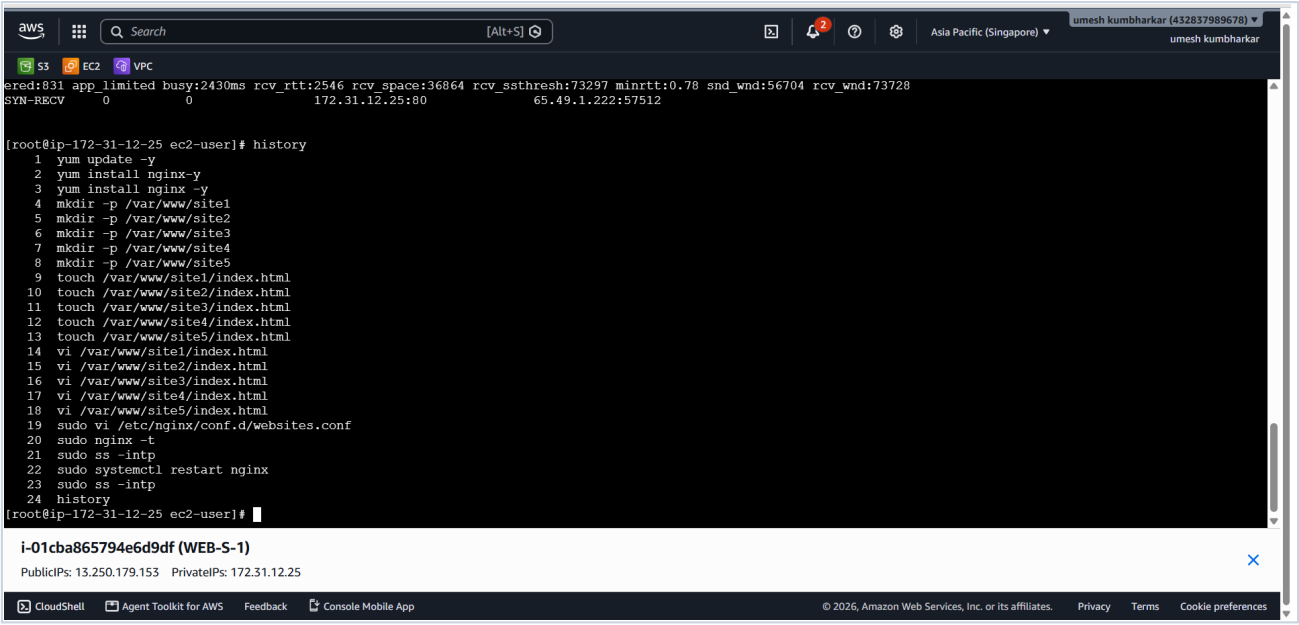
Website 4 - Port 9090 - http://13.250.179.153:9090 - "AWS Cloud Services"



Website 5 - Port 9091 - http://13.250.179.153:9091 - "Docker & Containers"

6. Command history from the server

This screenshot is the `history` command on the server. It shows every command we typed, in order.



Terminal: command history of WEB-S-1 (i-01cba865794e6d9df)

History line	Command	Purpose
1	yum update -y	Update the server
2 - 3	yum install nginx -y	Install Nginx (line 2 had a typo, line 3 is correct)
4 - 8	mkdir -p /var/www/site1 ... site5	Create 5 website folders
9 - 13	touch /var/www/siteN/index.html	Create 5 empty home pages
14 - 18	vi /var/www/siteN/index.html	Write the HTML content
19	sudo vi /etc/nginx/conf.d/websites.conf	Create the Nginx settings for 5 ports
20	sudo nginx -t	Test the settings
21	sudo ss -intp	Check ports (before restart)
22	sudo systemctl restart nginx	Restart Nginx
23	sudo ss -intp	Check ports (after restart)
24	history	Show all commands

Why do we use `history`?

history lists every command you typed before. It is very useful to prove what you did, to repeat a command, and to write documentation like this one.

7. Linux commands: what they mean and why we use them

Command	Meaning	Why we used it
`yum update -y`	Update all installed software	Fresh, secure server before installing anything
`yum install nginx -y`	Download and install Nginx	We need a web server to host websites
`mkdir -p`	Make a folder (and parents)	One folder per website in /var/www
`touch`	Create an empty file	Make the index.html home page files
`vi`	Terminal text editor	Write HTML and Nginx settings on a server with no GUI
`sudo`	Run as administrator	Protected folders and services need admin power
`nginx -t`	Test Nginx settings	Catch mistakes before restarting
`systemctl restart nginx`	Stop and start the service	Load the new settings and open the new ports
`systemctl enable nginx`	Start the service at every boot	Websites come back automatically after a reboot
`ss -intp`	Show network connections	Check that Nginx is listening on all 5 ports
`ls /var/www`	List files and folders	Confirm the 5 folders exist
`chmod -R 755`	Change file permissions	Fix a 403 Forbidden error (owner full access, others read)
`history`	Show typed commands	Record and review the work

8. If something does not work

Problem	Most likely reason	What to do
Page does not load (keeps loading)	Port not open in Security Group	EC2 > Security Groups > Inbound rules > add the port
Port 80 works, other ports do not	Only port 80 was allowed	Add 8080, 8081, 9090, 9091 to inbound rules
nginx -t shows an error	Typing mistake in websites.conf (missing ; or })	Open the file again, fix the line shown in the error
Nginx default page appears	Default server block in nginx.conf is taking port 80	Comment it out or use default_server, then test and restart
403 Forbidden	Wrong folder or file permission	Check the root path; run chmod -R 755 /var/www
Old page still shows	Browser cache	Use Incognito or press Ctrl + F5
Cannot connect after server restart	Public IP changed	Check the new IP, or attach an Elastic IP

9. Quick recap

#	What we did
1	Launched an EC2 instance and opened ports 22, 80, 8080, 8081, 9090, 9091 in the Security Group
2	Updated the server and installed Nginx
3	Created 5 folders and 5 index.html files
4	Wrote different HTML content in each file
5	Created websites.conf with 5 server blocks (one port per site)
6	Tested with nginx -t, restarted Nginx, and checked ports with ss
7	Opened all 5 websites in the browser using IP:port

10. Interview questions you can answer now

Q1. How can one server host many websites?
Nginx uses server blocks. Each block listens on a different port (or domain name) and points to a different folder.

Q2. Why must we run nginx -t before restart?
It finds mistakes in the settings file, so the running websites do not go down.

Q3. Websites work on the server but not in the browser. Why?
Usually the port is not allowed in the Security Group (or the Network ACL).

Q4. What is the difference between a public IP and a private IP?
The public IP (13.250.179.153) is reachable from the internet. The private IP (172.31.12.25) works only inside the AWS VPC.

Q5. Which command shows the ports that a server is listening on?
sudo ss -ltnp (or netstat -tulpn).

> **Next ideas**
> Attach an **Elastic IP** so the address never changes, add a **domain name** with Route 53, enable **HTTPS** with a free SSL certificate, and later put the server behind an **Application Load Balancer**.